

GSAcc: Accelerate 3D Gaussian Splatting via Depth Speculation and Gaussian-centric Rasterization

Mengtian Yang, Yipeng Wang, Chieh-Pu Lo, Xiuhao Zhang, Sirish Oruganti and Jaydeep P. Kulkarni

The University of Texas at Austin

{mengtian.yang, yipeng.wang, kcplo, zhxhao007, sirishoruganti}@utexas.edu, jaydeep@austin.utexas.edu

Abstract—3D Gaussian Splatting (3DGS) has emerged as a promising real-time photorealistic radiance field rendering technique. Existing GPU and hardware accelerators face limitations due to insufficient parallelism in sequential rendering pipeline stages and the memory overhead associated with interim results. This paper presents GSAcc, a hardware accelerator co-designed with dataflow to render compressed 3DGS models on edge platforms efficiently. GSAcc enhances 3DGS rendering performance through several key innovations. First, it introduces Gaussian depth speculation, parallelizing preprocessing and sorting tasks. Second, GSAcc adopts a Gaussian-centric dataflow that interleaves preprocessing and rasterization, allowing all rendering steps to execute concurrently without storing intermediate results. Finally, it employs dedicated hardware acceleration to address sorting and rasterization bottlenecks within the optimized dataflow. We implemented and synthesized GSAcc using Intel16 PDK and evaluated its performance on real-world 3DGS scenes. Compared with desktop GPUs, GSAcc achieves up to $1.66 \times 10^4 \times$ Power-Performance-Area (PPA) improvement as well as 48.7x energy savings. Additionally, GSAcc outperforms the state-of-the-art hardware accelerator GScore with up to 2.3x PPA improvement and 2.9x energy savings.

Index Terms—3D Gaussian Splatting, Software/Hardware Co-design, Accelerator Architecture, Radiance Field Rendering

I. INTRODUCTION

Neural Radiance Fields (NeRFs) [1] enable photorealistic image synthesis from novel viewpoints by learning continuous volumetric scene representations from images. Still, they suffer from significant computational demands and very high rendering latency. 3D Gaussian Splatting (3DGS) [2] has recently emerged as a substantial advancement in radiance field rendering, explicitly addressing the challenge of rendering speed and achieving higher image quality. This technique utilizes the mathematical characteristics of Gaussian functions to construct continuous and differentiable representations of 3D scenes. By projecting these 3D Gaussians onto image planes, the method can generate high-quality images with smoother transitions and fewer artifacts than traditional NeRF approaches. Additionally, when combined with advanced compression techniques [3], 3DGS achieves impressive rendering speeds on GPU hardware while maintaining a relatively compact model size. This balance of efficiency and quality makes it a promising solution for real-time radiance field rendering applications.

Despite its promising capabilities, 3DGS has substantial challenges in computational efficiency and memory requirements when deployed on AR/VR platforms, which demand real-time rendering performance within limited power and memory constraints. The original 3DGS algorithm contains three interdependent steps, making it difficult to parallelize and pipeline the processes effectively. Additionally, each step requires the

computation of intermediate results for millions of Gaussians, which must be stored for subsequent processing. This necessity leads to significant memory overhead, further complicating the deployment of 3DGS in resource-constrained AR/VR environments. This paper introduces GSAcc, a compressed 3DGS accelerator designed to optimize the 3DGS under the edge platform.

In summary, this paper makes the following contributions:

- First, we analyze the rendering pipeline of 3DGS in a real-time model viewer and propose a depth speculation technique based on previous frames to parallelize sorting.
- Second, we introduce a Gaussian-centric dataflow that interleaves pre-processing and rasterization, eliminating the need for storing intermediate results.
- Third, we design and implement GSAcc, a hardware accelerator that supports the optimized dataflow. GSAcc achieves up to $1.66 \times 10^4 \times$ PPA improvement and 48.7x energy savings compared with GPU platform and up to 2.3x PPA improvement and 2.9x energy savings compared with state-of-the-art accelerator GScore [4].

II. BACKGROUND

In this section, we will introduce the background of 3DGS. We will first explain how to store scene information in 3D Gaussians and then illustrate how to render a scene from the model.

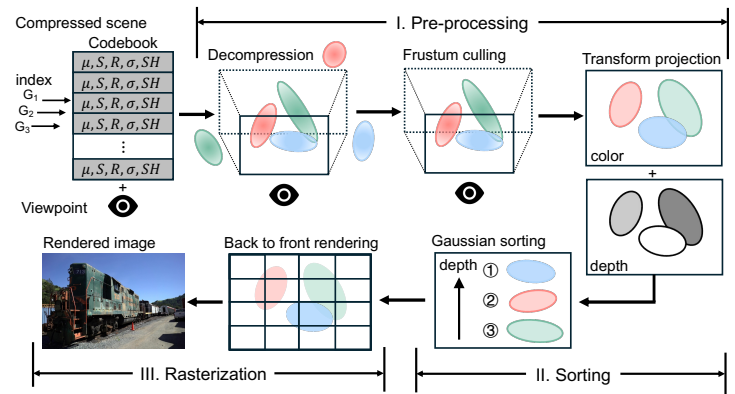


Fig. 1: The Pre-processing, Sorting, and Rasterization pipeline for compressed 3D Gaussian Splatting model rendering.

A. 3DGS Model with Compression

In a 3DGS model, scenes are explicitly represented as ensembles of three-dimensional Gaussian primitives. Each Gaussian has five key attributes: position μ represents the center of the Gaussian, scale S describes the size of the Gaussian along each axis,

rotation R specifies the orientation of the Gaussian, opacity σ indicates the transparency level, and color SH defines the color characteristics encoded using sphere harmonics. Each Gaussian primitive effectively models a localized region of the scene, and the entire scene is constructed to balance geometric fidelity and visual realism by aggregating millions of Gaussians.

Since each scene usually contains many Gaussians, the vanilla 3DGS model is often much larger than a NeRF model for the same scene. To effectively reduce the memory footprint and enhance the computational efficiency of 3DGS models, vector quantization using k-means clustering presents a robust compression strategy [3]. This approach involves partitioning the high-dimensional parameter space of Gaussian primitives, including position, scale, rotation, color, and opacity, into a finite set of representative centroids through the k-means algorithm. By clustering similar Gaussian primitives together, each Gaussian in the original model can be approximated by the nearest centroid from the learned codebook, thereby significantly reducing the number of parameters that need to be stored.

B. Rendering Pipeline

Fig. 1 illustrates the rendering pipeline from the viewpoint of an image within a compressed 3DGS model. This pipeline contains three sequential and interdependent stages: pre-processing, sorting, and rasterization. Intermediate results from each stage are stored in memory for the next stage.

Pre-processing: In the pre-processing stage, each Gaussian is firstly decompressed from the codebook to retrieve its parameters. Once the parameters are obtained, each Gaussian is evaluated to determine whether it falls within the viewing frustum—the region of space potentially visible on the final screen. Based on their depth information, Gaussians outside this frustum are excluded through a process known as frustum culling. Subsequently, the depth of each Gaussian is calculated, and a transformation projection is applied to map the 3D Gaussians onto the 2D screen space. Also, the color of each Gaussian is computed based on its spherical harmonics representation, ensuring accurate color representation during rendering.

Sorting: Following pre-processing, the Gaussians projected onto the 2D screen space are organized based on their depth and color attributes. Since Gaussians closer to the viewpoint can shield those farther away, depth-based sorting is essential to maintain the correct rendering image. All Gaussians are sorted in ascending order of their depth values, from nearest to farthest, and their ordered indices are generated to facilitate the subsequent rendering step.

Rasterization: In the rasterization stage, the projected and sorted Gaussians are rendered into the final image. The screen is divided into tiles with 16x16 pixels in each tile, and rendering is performed parallel across tiles to optimize performance. For each pixel, the relevant Gaussians are sampled to determine the pixel opacity (α), and alpha-blending [5] is used to calculate the final color of the pixel. This rendering process proceeds from the back to the front based on the depth information of the Gaussians, ensuring proper occlusion and shielding effects.

III. MOTIVATION AND ANALYSIS

This section explores the potential for reusing information from the previous frame to enable parallelism between the sorting step and other processing stages. We then analyze the current rasterization pipeline to identify opportunities for parallelizing pre-processing and rasterization by leveraging data from the last frame. Finally, we examine the tiling process during rasterization to uncover possibilities for optimizing memory access.

A. Last Frame Information Reuse

Immersive applications are the primary use case for 3DGS. As illustrated in Fig. 2(a), camera movements and rotations result in changes to the rendered image when viewing a 3DGS model in immersive applications. At a reasonable frames per second (FPS), consecutive frames typically show only minor differences. This temporal coherence suggests that certain information from the previous frame, such as Gaussian depth and sorting results, can be reused in the rendering process for the current frame.

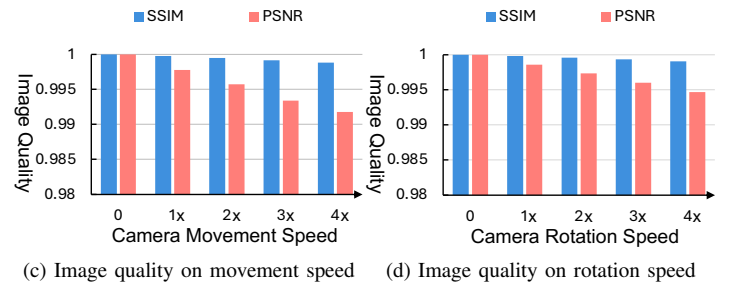
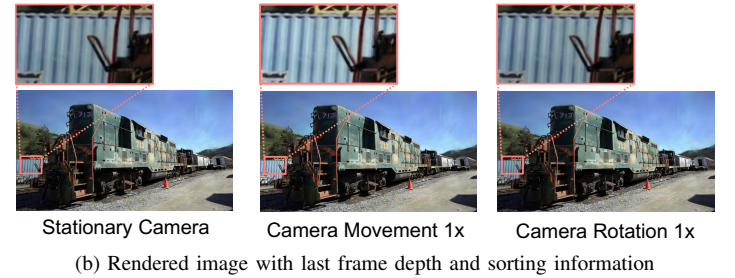
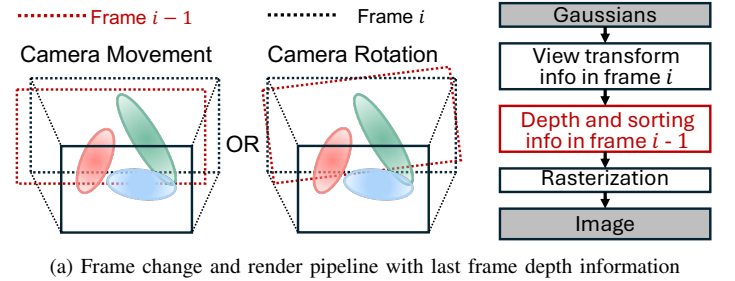


Fig. 2: Image quality analysis leveraging depth information from the previous frame. View transformation is based on current camera data, while Gaussian depth and sorting utilize the previous frame’s camera information.

We evaluate the rendered image quality by leveraging depth information and sorting orders from the previous frame under various camera movement and rotation speeds. Fig. 2 (b) shows the rendered image when using the depth from the previous frame. Fig. 2 (c) and (d) present the results of this analysis. The standard

movement speed (0.03 per frame) and rotation speed (0.5 degrees per frame) are based on the interactive viewer from the 3DGS paper [2] and represented as “1x”, while a speed of 0 indicates a stationary camera, serving as the ground truth for viewing. To test the robustness of our proposed approach, we accelerate camera movement and rotation speeds up to four times the standard speed. Image quality is evaluated using SSIM and PSNR metrics compared to ground truth. The analysis is performed in four different scenes and averaged. The results show that even at four times the standard camera speed, the degradation of image quality remains within 1% when using Gaussian depth and the sorting results of the previous frame.

B. Tile-centric vs Gaussian-centric Rasterization

The rasterization stage calculates the color of each screen pixel based on the intersecting Gaussians from back to front. Consequently, the rendering can either iterate through all Gaussians for each tile before moving to the next tile, as shown in Fig. 3(a). We refer to it as the *tile-centric* rasterization dataflow. Alternatively, rendering can prioritize Gaussians first and compute colors for all pixels across intersecting tiles, as shown in Fig. 3(b), and we call it *Gaussian-centric* rasterization dataflow.

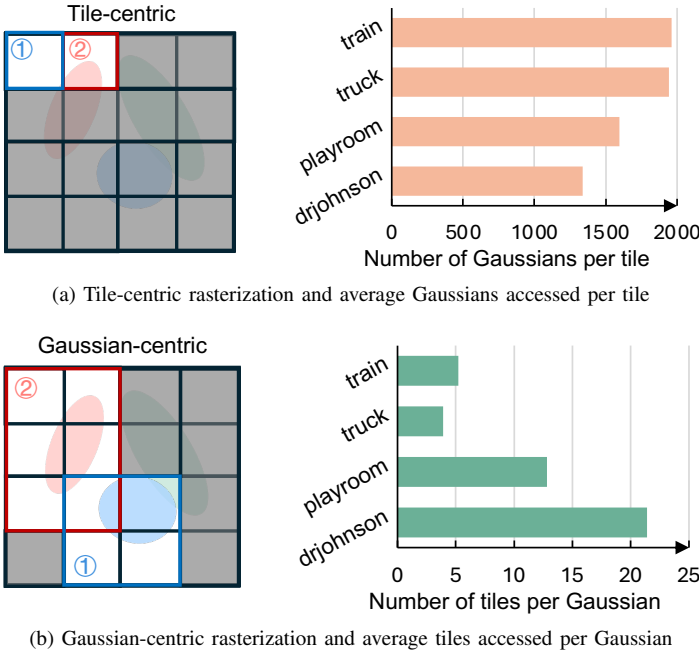


Fig. 3: Comparison of tile-centric rasterization with Gaussian-centric rasterization.

The right side of Fig. 3 illustrates the average number of Gaussians accessed per tile in the tile-centric dataflow compared to the average number of tiles accessed per Gaussian in Gaussian-centric dataflow. It is evident that each tile requires significantly more Gaussians than the number of tiles each Gaussian interacts with. In tile-centric dataflow, all accessed Gaussians must be ready before rasterizing a tile, aligning with the vanilla 3DGS rendering pipeline in Fig. 1, where pre-processing handles and stores all Gaussian information beforehand. However, by utilizing Gaussian depth and order information from the previous frame,

pre-processing and rasterization can be interleaved. In this scenario, Gaussian-centric dataflow is preferable, as it eliminates the need to store interim pre-processing results and simplifies tile management, with each Gaussian typically affecting only a few tiles on average.

C. Tile Stationary Across Gaussians

In Gaussian-centric dataflow, Gaussians are rasterized in ascending order of their depth. As a result, subsequent Gaussians may be positioned close to the current one, leading to overlapping tiles. To minimize DRAM access, the color information for these overlapping tiles should be retained on-chip as much as possible.

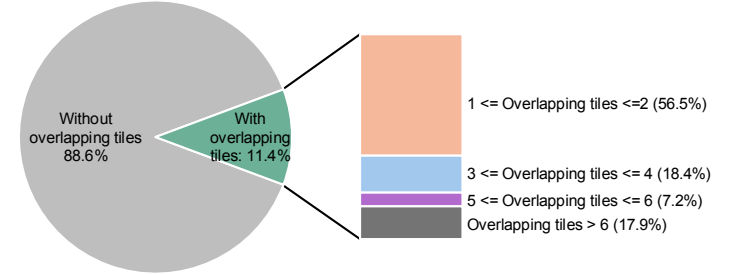


Fig. 4: Overlapping ratio of tiles for current Gaussian and tiles for previous Gaussian with Gaussian-centric rasterization.

Fig. 4 illustrates the ratio of overlapping to non-overlapping tiles between two adjacent Gaussians during rasterization with Gaussian-centric dataflow, averaged across four different benchmarks. The results show that approximately 11.4% of tiles overlap with those of the preceding Gaussian. Additionally, the majority of these overlaps occur on four or fewer tiles, indicating that a relatively small on-chip buffer is sufficient to optimize tile stationary effectively.

IV. PROPOSED DESIGN

This section illustrates the proposed GSAcc, a 3D Gaussian Splatting accelerator architecture co-designed with depth speculation and Gaussian-centric dataflow. We will first introduce the architecture design and each hardware unit, followed by the proposed fully parallelized dataflow.

A. Overall Architecture

Fig. 5 illustrates the overall architecture of our proposed GSAcc design. GSAcc comprises three primary modules: the Gaussian pre-processing module, which calculates transformations, projections, opacity, and color for each Gaussian; the Gaussian culling and sorting module, which performs frustum culling, computes Gaussian depths, and sorts the Gaussians based on depth; and the rasterization module, which determines the screen color for all pixels by processing the sorted Gaussians. These modules are interconnected via a bus and interface with off-chip memory. Additionally, GSAcc features a global shared buffer that caches Gaussian parameters from the compressed codebook. Both the Gaussian sorting module and the rasterization module are equipped with dedicated buffers, enhancing performance by leveraging data locality.

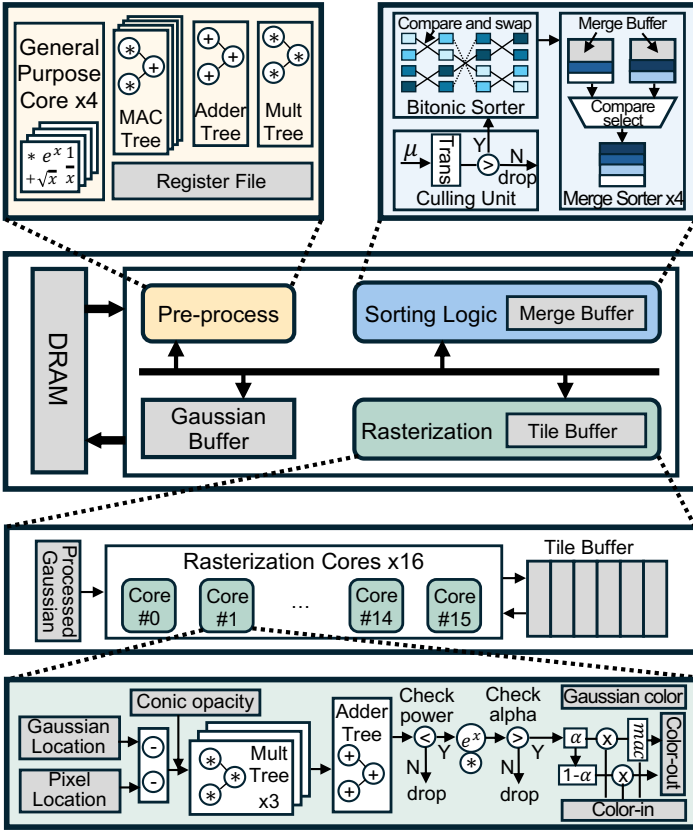


Fig. 5: Overall architecture of the proposed GSAcc. It contains a pre-processing module, a sorting module with a dedicated merge buffer, a rasterization module with a dedicated tile buffer, and a global shared Gaussian buffer.

B. Pre-processing Module

The pre-processing of a Gaussian takes the Gaussian index from the compressed codebook and camera information as input and calculates the corresponding color, conic opacity, and the tiles affected during rasterization. Since most operations involve three-dimensional points or four-dimensional quaternions, we design the pre-processing components to efficiently handle these operations. The module includes four multiply-accumulate (MAC) units, each accepts two 4-dimensional vectors and performs vector dot product. These MAC units accelerate tasks such as 3D covariance generation, view transformation, and 3D-to-2D projection, which require matrix-matrix and matrix-vector multiplications at a granularity of three or four operands per dimension. The module also features a four-input adder tree and a four-input multiplier tree to speed up continuous addition and multiplication operations in color computation from spherical harmonics. Other miscellaneous pre-processing tasks are managed by four general-purpose cores, which incorporate various arithmetic logic units, including addition, multiplication, square root, exponential functions, min/max logic, and reciprocal operations.

C. Culling and Sorting Module

The culling and sorting module of GSAcc is designed to calculate the depth of each Gaussian based on its mean location μ and the camera information represented by a view transform matrix. This module filters out Gaussians that lie outside the view frustum and

sorts the remaining in-frustum Gaussians in descending order of their depth. Since in-frustum culling relies solely on the depth information of each Gaussian, the module only requires one MAC tree and one comparator to compute the z-dimension of the view transformation and perform the culling efficiently.

Sorting is a critical step in 3DGS rendering and has been a bottleneck in the original rendering pipeline. However, with the co-designed dataflow, rasterization is no longer dependent on the sorting results of the current frame, allowing us to prioritize efficiency and memory access when designing the sorting unit. We propose a two-stage accurate sorter in GSAcc. The first stage utilizes a 32-way bitonic sorter [6] that directly receives data from the culling unit. Once it collects 32 Gaussians, it begins the sort operation, comparing and swapping Gaussian pairs based on depth in a hierarchical manner. Since the compare-and-swap logic is fixed in hardware, the bitonic sorter is fully pipelined and achieves high throughput. The second stage consists of four merge sorters, each takes two sorted blocks as input and merges them into a single sorted block. This stage is efficiently implemented in hardware with compare-and-select logic, producing one sorted result per cycle per sorter. To minimize external memory accesses, the merge sorter is equipped with four dedicated merge buffers with a depth of 1024. Output chunks from the bitonic sorter are stored in two of the buffers, and the merged results remain on-chip until the buffer reaches its maximum capacity. At that point, the merged results are offloaded to DRAM, and a new merge iteration begins. Once all Gaussians are merged into large chunks matching the output buffer capacity, the external merge sorting algorithm is used for the remaining merge sorting steps.

D. Rasterization Module

The rasterization module of GSAcc is a critical component for performance, as rasterization must handle the total number of in-frustum Gaussians multiplied by the average number of pixels per Gaussian, which is typically several hundred times higher than the data processed by sorting and pre-processing modules. To maximize rasterization performance, the entire rasterization workflow is handled by a dedicated hardware block with a fully pipelined design in GSAcc. The module features 16 rasterization cores, each capable of rendering one pixel per cycle for the same Gaussian inside a tile with 16x16 pixels, achieving a maximum throughput of up to 16 pixels per cycle. Additionally, GSAcc includes one input buffer, one output buffer, and four extra stationary buffers to support tile-stationary scheduling.

Each rasterization core processes the locations, colors, and opacity of both screen pixels and the current Gaussian. The core first computes the relative Gaussian location by subtracting the pixel location from the original Gaussian location. It then generates the Gaussian power value based on the relative location and Gaussian opacity through several multiplications and additions. To enhance parallelism and reduce latency, each core is equipped with three 4-input multiplier trees and one 4-input adder tree, enabling the Gaussian power calculation to proceed in parallel. Following this, a comparator filters out invalid Gaussians by checking their power values. An exponential logic unit, combined with a multiplier, calculates the opacity contribution of the current Gaussian to the pixel. Gaussians with relatively low opacity are

discarded as they contribute minimally to the pixel color. Finally, the screen color and opacity are updated using the Gaussian color and opacity with the alpha blending algorithm.

E. Fully Parallelized Dataflow

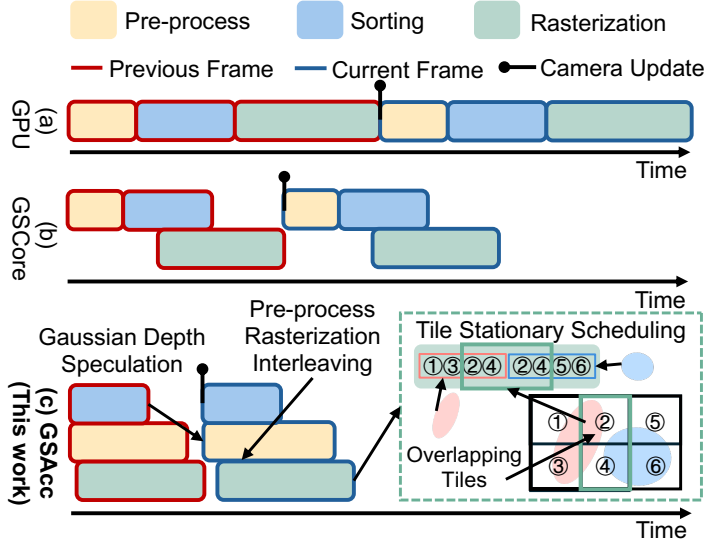


Fig. 6: Execution timeline for different dataflows in rendering for a 3DGS viewer. (a) Original GPU rendering dataflow. (b) GSCore dataflow with interleaved sorting/rasterization. (c) Proposed fully parallelized GSAcc dataflow, enabling pre-processing, sorting, and rasterization to execute concurrently.

Figure 6 illustrates the execution timeline for existing dataflows on the GPU platform [3] and the previous work GSCore, alongside our proposed dataflow for GSAcc when rendering continuous frames in a 3DGS viewer. As described in Fig. 6(a), the original rendering pipeline for a 3DGS scene on the GPU platform processes frames sequentially. Each frame undergoes pre-processing, sorting, and rasterization to produce the rendered image, with no overlap or parallelism between these steps. Fig. 6(b) shows the execution dataflow of GSCore. GSCore splits the sorting into two pipelined stages: first, it sorts all Gaussians into multiple chunks, and then it performs precise sorting within each chunk. Rasterization can start after the first chunk is precisely sorted, allowing partial overlap and parallel execution of sorting and rasterization. However, GSCore still needs to wait for the pre-processing to complete after a camera update.

Our proposed dataflow for GSAcc is illustrated in Fig. 6(c). It introduces Gaussian depth speculation by reusing the depth and order information from the previous frame as the basis for sorting in the current frame. After a camera update, GSAcc simultaneously initiates sorting and pre-processing using the updated camera information. By leveraging this depth and order speculation, Gaussians are pre-processed with depth from the last frame and view transformations and projections from the current frame. This allows rasterization to begin right after the first Gaussian is pre-processed, enabling the sorting, pre-processing, and rasterization stages to interleave and operate in parallel until all Gaussians are processed. Thus, our dataflow effectively overlaps and parallelizes these three stages, significantly enhancing rendering efficiency.

To further optimize memory access, we propose a tile-stationary aware dataflow that tracks overlapping tiles between consecutive Gaussians and adjusts the rasterization order accordingly. This ensures that overlapping tiles remain in tile buffers as much as possible during Gaussian transitions, thereby enhancing memory efficiency and reducing latency.

V. EVALUATION

A. Methodology and Experiment Setup

To evaluate the performance of GSAcc, we develop an architectural simulator that models the number of cycles and off-chip memory accesses. We use the same LPDDR4-3200 as the GSCore with a maximum DRAM bandwidth of 51.2GB/s to ensure a fair comparison. Similar to GSCore, DRAM access time is simulated using Ramulator [7] and DRAM power is simulated with DRAMPower [8]. To accurately estimate on-chip power and area, we implement the RTL design in SystemVerilog. The design is synthesized with Cadence Genus using Intel16 technology under a 500MHz frequency constraint. The floating-point arithmetic units are sourced from Cadence Chipware IPs, and the buffer SRAMs are generated with an SRAM compiler from Intel. For benchmarks, we select two outdoor scenes and two indoor scenes from two datasets: “Train” and “Truck” from Tanks & Temples [9], and “Playroom” and “Drjohnson” from Deep Blending [10]. These trained models were obtained from CompGS [3], and the 3D Gaussian parameters are also quantized to FP16 for a fair comparison with GSCore.

B. Compare with Existing Design

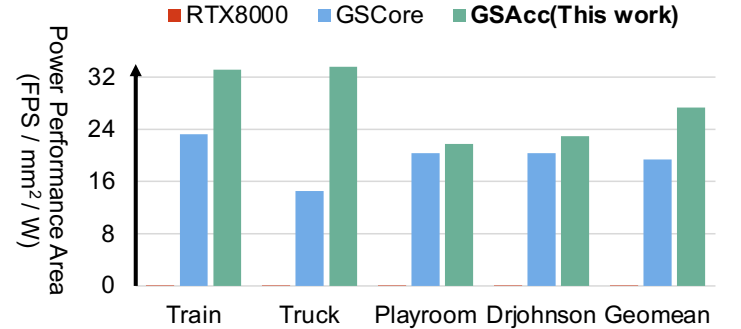


Fig. 7: Power Performance Area (PPA) comparison results of GSAcc over GPU and GSCore.

We compare our GSAcc design against both RTX8000 GPU platform and the state-of-the-art accelerator platform GSCore. To effectively evaluate the performance of different designs across varying technologies and scales, we consider Power-Performance-Area (PPA) as the primary comparison metric. Consequently, we utilize rendering frames per second per square millimeter per watt (FPS/mm²/W) as our key metric. Fig. 7 illustrates the PPA results achieved by GSAcc. Results for the GPU platform are profiled with a code base from Compact3D and the results for GSCore are directly referenced from the original paper. When comparing to the GPU platform, GSAcc achieves up to 1.66×10^4 x higher PPA and an average improvement of 1.35×10^4 x. Additionally, GSAcc outperforms the state-of-the-art accelerator GSCore by achieving up to 2.30x higher PPA and an average of 1.41x higher PPA.

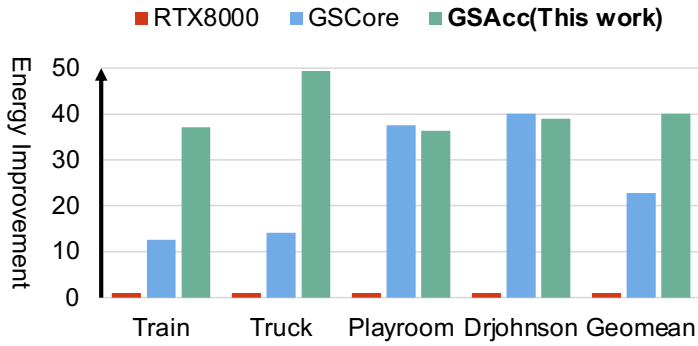


Fig. 8: Energy improvement results of GSAcc over GPU and GScore.

In terms of energy efficiency improvement, we calculate the energy consumed to render a single frame and then determine the factor by which energy usage is reduced compared to the GPU platform. Figure 8 presents the comparison results for GSAcc. GSAcc can save up to 48.7x energy per frame compared to the GPU platform and up to 2.9x energy per frame compared to GScore. On average, GSAcc achieves a 39.8x improvement in energy efficiency over the GPU platform and a 1.74x improvement compared to GScore.

C. Performance Improvement Analysis

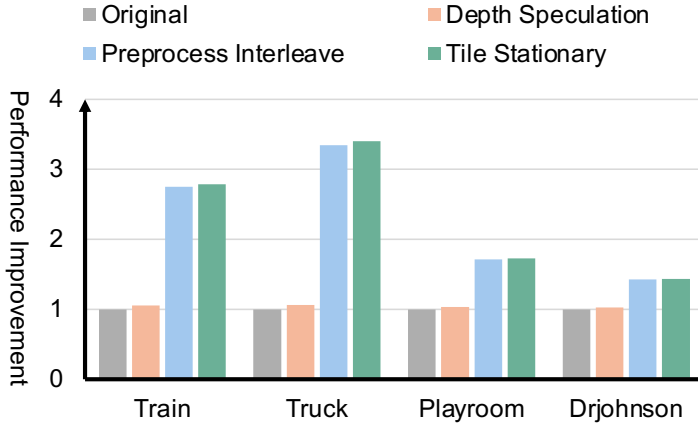


Fig. 9: Breakdown of GSAcc performance improvement with each optimization technique.

To further investigate the contribution of each optimization method applied in GSAcc, we first ran the original dataflow on the GSAcc architecture to establish a performance baseline, as shown in Fig. 9. We then introduce Gaussian depth speculation, enabling parallel execution of the sorting process. This enhancement results in a performance improvement of 4%–5%, primarily because the sorter module in GSAcc is optimized. Next, by incorporating pre-processing overlapping, we observe a significant performance increase across all scenes, as pre-processing and rasterization could operate concurrently without storing intermediate results. Finally, implementing tile stationary adds an additional 5% performance boost and reduces memory access energy by decreasing off-chip memory accesses. These incremental optimizations collectively demonstrate the effectiveness of our design choices in enhancing GSAcc’s overall performance and energy efficiency.

D. Area/Power Breakdown

TABLE I: Synthesized area and power of GSAcc submodules

Module	Area (mm ²)	Power (W)
Pre-processing module	0.0507	0.0297
Culling and sorting module	0.1006	0.1206
Rasterization module	0.1377	0.2395
SRAMs + Others	0.1023	0.0187
Total	0.3913	0.4085

Table I presents the synthesized area and power results for GSAcc. Our GSAcc architecture consumes a total of 0.4085W and occupies an area of 0.3913mm². The rasterization module is the primary contributor to both area and power consumption, highlighting its role as the key high-performance component. Additionally, the sorter module, particularly the bitonic sorter, significantly impacts both power and area. In contrast, SRAM buffers occupy a large portion of the area but contribute minimally to power consumption.

VI. RELATED WORK

Given the exceptional performance of 3DGS and its promising potential in AR/VR applications, efforts to accelerate 3DGS have captured considerable attention within the algorithmic community. Several studies have demonstrated the feasibility of compressing Gaussian models to reduce their size [11]–[13], thereby enhancing rendering speed on GPUs by lowering memory bandwidth requirements. Additionally, techniques such as Gaussian opacity regulation [14] have been developed to accelerate rendering by decreasing the number of Gaussians during model training, further improving efficiency.

On the system and architecture front, researchers have begun exploring lower-level acceleration opportunities for 3DGS. For instance, FlashGS [15] provides an optimized CUDA library specifically designed for accelerating 3DGS rendering on GPU platforms, while GScore [4] introduces a dedicated hardware accelerator tailored for this purpose. However, these approaches have not yet achieved a fully parallelized rendering pipeline. Moreover, they have not investigated the utilization of information from previous frames to enhance rendering performance in real-time AR/VR applications.

VII. CONCLUSION

This paper proposed GSAcc, an architecture/dataflow co-design solution to accelerate 3D Gaussian Splatting. GSAcc effectively leverages frame-to-frame information reuse and optimizes the rendering pipeline through parallelized sorting and rasterization, significantly enhancing performance and energy efficiency compared to existing GPU and accelerator platforms. Our comprehensive evaluations demonstrate that GSAcc delivers substantial improvements in PPA and energy efficiency, making it a viable solution for immersive and resource-constrained environments.

ACKNOWLEDGMENT

We thank Intel for PDK and memory compiler support, Cadence for IP and EDA tools support, and the UT iMAGINE consortium for supporting this research project.

REFERENCES

- [1] B. Mildenhall, P. P. Srinivasan, M. Tancik, J. T. Barron, R. Ramamoorthi, and R. Ng, "Nerf: representing scenes as neural radiance fields for view synthesis," *Commun. ACM*, vol. 65, no. 1, p. 99–106, Dec. 2021. [Online]. Available: <https://doi.org/10.1145/3503250>
- [2] B. Kerbl, G. Kopanas, T. Leimkühler, and G. Drettakis, "3d gaussian splatting for real-time radiance field rendering," *ACM Transactions on Graphics*, vol. 42, no. 4, July 2023. [Online]. Available: <https://repo-sam.inria.fr/fungraph/3d-gaussian-splatting/>
- [3] K. Navaneet, K. P. Meibodi, S. A. Koohpayegani, and H. Pirsiavash, "Compgs: Smaller and faster gaussian splatting with vector quantization," 2024. [Online]. Available: <https://arxiv.org/abs/2311.18159>
- [4] J. Lee, S. Lee, J. Lee, J. Park, and J. Sim, "Gscore: Efficient radiance field rendering via architectural support for 3d gaussian splatting," in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, ser. ASPLOS '24. New York, NY, USA: Association for Computing Machinery, 2024, p. 497–511. [Online]. Available: <https://doi.org/10.1145/3620666.3651385>
- [5] B. A. Wallace, "Merging and transformation of raster images for cartoon animation," *SIGGRAPH Comput. Graph.*, vol. 15, no. 3, p. 253–262, Aug. 1981. [Online]. Available: <https://doi.org/10.1145/965161.806813>
- [6] T. Nakatani, S.-T. Huang, B. Arden, and S. Tripathi, "K-way bitonic sort," *IEEE Transactions on Computers*, vol. 38, no. 2, pp. 283–288, 1989.
- [7] Y. Kim, W. Yang, and O. Mutlu, "Ramulator: A fast and extensible dram simulator," *IEEE Computer Architecture Letters*, vol. 15, no. 1, pp. 45–49, 2016.
- [8] K. Chandrasekar, C. Weis, Y. Li, S. Goossens, M. Jung, O. Naji, B. Akesson, N. Wehn, and K. Goossens, "Drampower: Open-source dram power & energy estimation tool," <http://www.drampower.info>.
- [9] A. Knapitsch, J. Park, Q.-Y. Zhou, and V. Koltun, "Tanks and temples: benchmarking large-scale scene reconstruction," *ACM Trans. Graph.*, vol. 36, no. 4, Jul. 2017. [Online]. Available: <https://doi.org/10.1145/3072959.3073599>
- [10] P. Hedman, J. Philip, T. Price, J.-M. Frahm, G. Drettakis, and G. Brostow, "Deep blending for free-viewpoint image-based rendering," *ACM Trans. Graph.*, vol. 37, no. 6, Dec. 2018. [Online]. Available: <https://doi.org/10.1145/3272127.3275084>
- [11] S. Niedermayr, J. Stumpfegger, and R. Westermann, "Compressed 3d gaussian splatting for accelerated novel view synthesis," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2024, pp. 10 349–10 358.
- [12] Z. Fan, K. Wang, K. Wen, Z. Zhu, D. Xu, and Z. Wang, "Lightgaussian: Unbounded 3d gaussian compression with 15x reduction and 200+ fps," 2024. [Online]. Available: <https://arxiv.org/abs/2311.17245>
- [13] M. T. Bagdasarian, P. Knoll, Y.-H. Li, F. Barthel, A. Hilsmann, P. Eisert, and W. Morgenstern, "3dgs.zip: A survey on 3d gaussian splatting compression methods," 2024. [Online]. Available: <https://arxiv.org/abs/2407.09510>
- [14] A. Guédon and V. Lepetit, "Sugar: Surface-aligned gaussian splatting for efficient 3d mesh reconstruction and high-quality mesh rendering," *CVPR*, 2024.
- [15] G. Feng, S. Chen, R. Fu, Z. Liao, Y. Wang, T. Liu, Z. Pei, H. Li, X. Zhang, and B. Dai, "Flashgs: Efficient 3d gaussian splatting for large-scale and high-resolution rendering," 2024. [Online]. Available: <https://arxiv.org/abs/2408.07967>